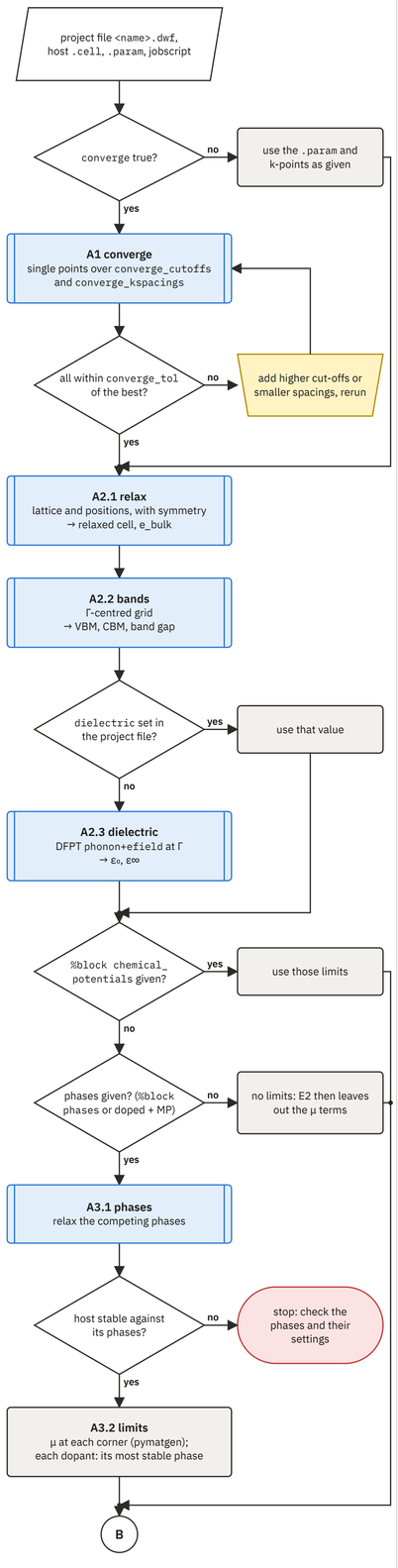
Defect workflow, as the scripts run it

One column per phase and script, top to bottom; each diamond is a decision and every branch out of it is labelled. Run the scripts in order with the same project file: `python3 dwf_host.py <name>.dwf`, then dwf_candidates, dwf_screen, dwf_production, dwf_thermo.

start / end · step done by the script · CASTEP jobs (supercell_convergence.py) · MACE (charge-blind; runs where the script runs) · ShakeNBreak · your action · decision · input · B on to the next phase

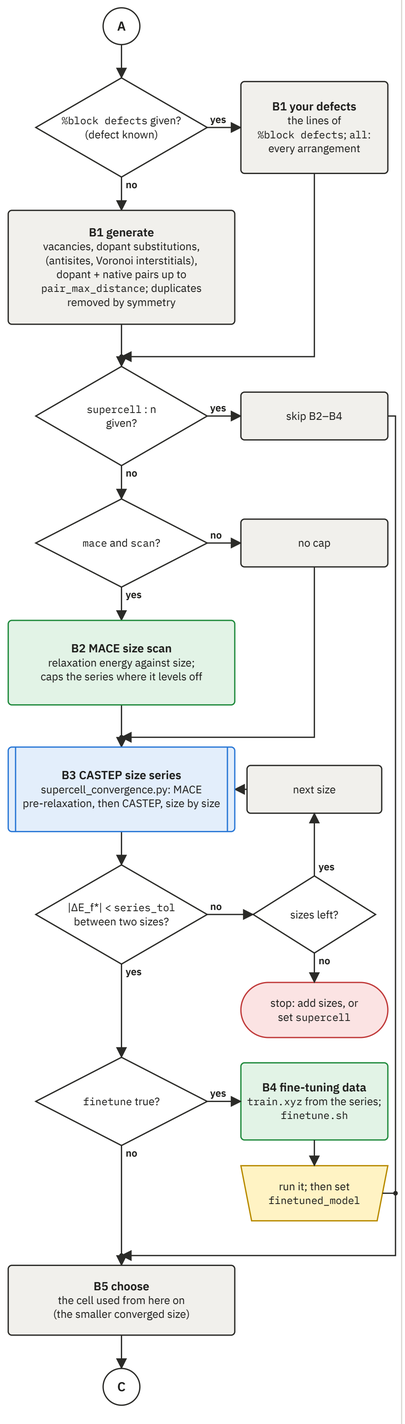
A Host reference

`dwf_host.py` · once per host



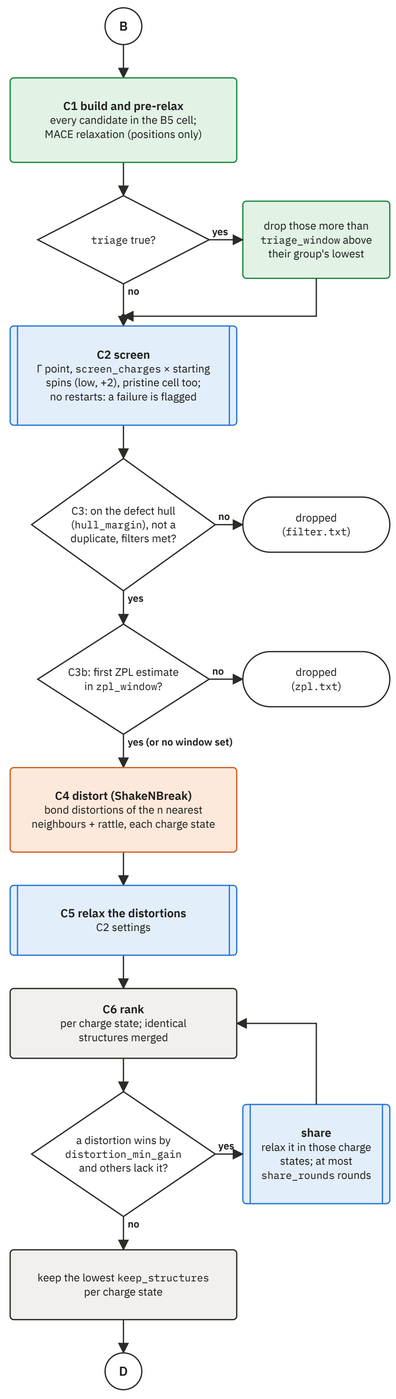
B Candidates and cell size

`dwf_candidates.py` · once per host



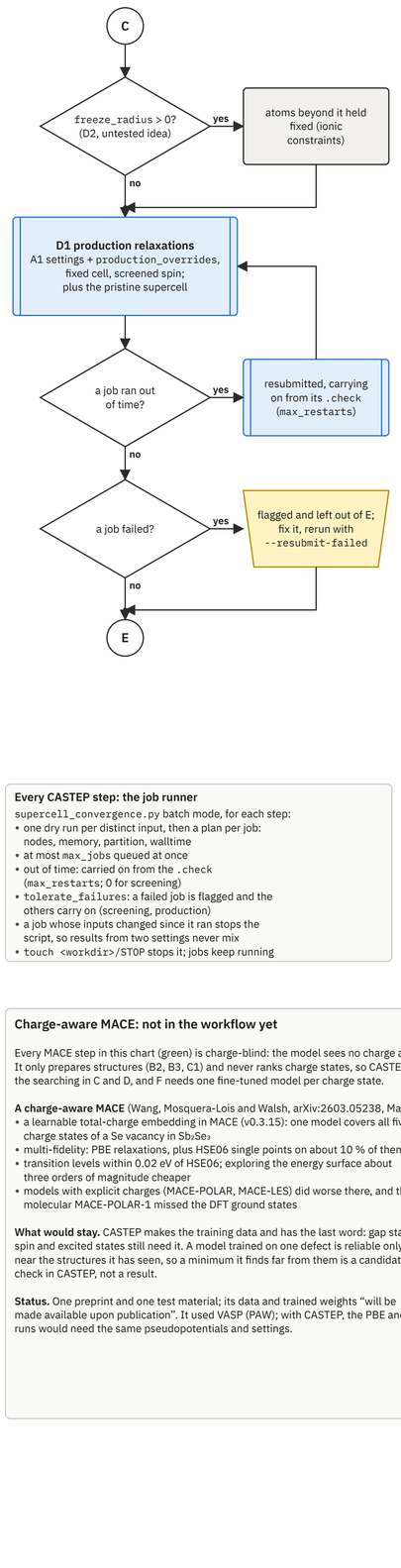
C Screening and ground states

`dwf_screen.py` · every candidate, then survivors



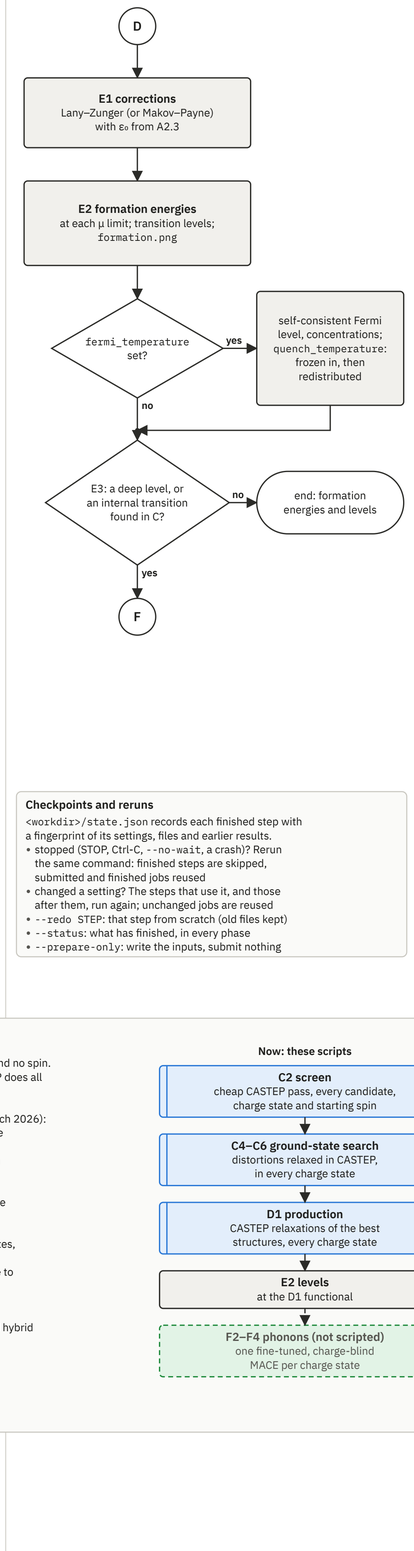
D Production relaxations

`dwf_production.py` · survivors × charge states



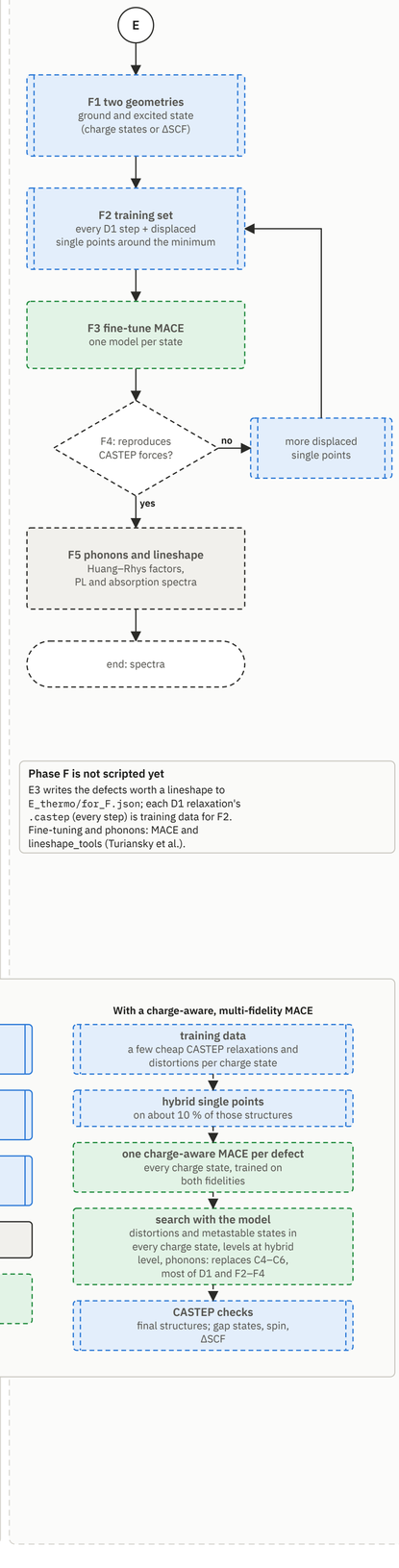
E Thermodynamics

`dwf_thermo.py` · per defect



F Optical lineshape

not scripted yet · selected defects



Every CASTEP step: the job runner

`supercell_convergence.py` batch mode, for each step:

- one dry run per distinct input, then a plan per job: nodes, memory, partition, walltime
- at most `max_jobs` queued at once
- out of time: carried on from the `.check` (`max_restarts`; 0 for screening)
- `tolerate_failures`: a failed job is flagged and the others carry on (screening, production)
- a job whose inputs changed since it ran stops the script, so results from two settings never mix
- `touch <workdir>/STOP` stops it; jobs keep running

Checkpoints and reruns

`<workdir>/state.json` records each finished step with a fingerprint of its settings, files and earlier results.

- stopped (STOP, Ctrl-C, `--no-wait`, a crash)? Rerun the same command: finished steps are skipped, submitted and finished jobs reused
- changed a setting? The steps that use it, and those after them, run again; unchanged jobs are reused
- `--redo STEP`: that step from scratch (old files kept)
- `--status`: what has finished, in every phase
- `--prepare-only`: write the inputs, submit nothing

Phase F is not scripted yet

E3 writes the defects worth a lineshape to `E_thermo/for_F.json`; each D1 relaxation's `.castep` (every step) is training data for F2. Fine-tuning and phonons: MACE and lineshape_tools (Turiansky et al.).

Charge-aware MACE: not in the workflow yet

Every MACE step in this chart (green) is charge-blind: the model sees no charge and no spin. It only prepares structures (B2, B3, C1) and never ranks charge states, so CASTEP does all the searching in C and D, and F needs one fine-tuned model per charge state.

A charge-aware MACE (Wang, Mosquera-Lois and Walsh, arXiv:2603.05238, March 2026):

- a learnable total-charge embedding in MACE (v0.3.15): one model covers all five charge states of a Se vacancy in Sb_2Se_3
- multi-fidelity: PBE relaxations, plus HSE06 single points on about 10 % of them
- transition levels within 0.02 eV of HSE06; exploring the energy surface about three orders of magnitude cheaper
- models with explicit charges (MACE-POLAR, MACE-LES) did worse there, and the molecular MACE-POLAR-1 missed the DFT ground states

What would stay. CASTEP makes the training data and has the last word: gap states, spin and excited states still need it. A model trained on one defect is reliable only near the structures it has seen, so a minimum it finds far from them is a candidate to check in CASTEP, not a result.

Status. One preprint and one test material; its data and trained weights "will be made available upon publication". It used VASP (PAW); with CASTEP, the PBE and hybrid runs would need the same pseudopotentials and settings.

